

Test your knowledge of clustering and collaborative filtering

1. In this question, we will use cluster-then-predict, a methodology in which you first cluster observations and then build cluster-specific prediction models. In this assignment, we'll use cluster-then-predict to predict future stock prices using historical stock data. When selecting which stocks to invest in, investors seek to obtain good future returns. In this problem, we will first use clustering to identify clusters of stocks that have similar returns over time. Then, we'll use logistic regression to predict whether or not the stocks will have positive future returns. For this problem, we'll use `StocksCluster.csv`, which contains monthly stock returns from the NASDAQ stock exchange. The NASDAQ is the second-largest stock exchange in the world, and it lists many technology companies. The stock price data used in this problem was obtained from `infocimps`, a website providing access to many datasets. Each observation in the dataset is the monthly returns of a particular company in a particular year. The years included are 2000-2009. The companies are limited to tickers that were listed on the exchange for the entire period 2000-2009, and whose stock price never fell below \$1. So, for example, one observation is for Yahoo in 2000, and another observation is for Yahoo in 2001. Our goal will be to predict whether or not the stock return in December will be positive, using the stock returns for the first 11 months of the year. This dataset contains the following variables:

- **ReturnJan:** the return for the company's stock during January (in the year of the observation)
- **ReturnFeb:** the return for the company's stock during February (in the year of the observation)
- **ReturnMar:** the return for the company's stock during March (in the year of the observation)
- **ReturnApr:** the return for the company's stock during April (in the year of the observation)
- **ReturnMay:** the return for the company's stock during May (in the year of the observation)
- **ReturnJune:** the return for the company's stock during June (in the year of the observation)
- **ReturnJuly:** the return for the company's stock during July (in the year of the observation)
- **ReturnAug:** the return for the company's stock during August (in the year of the observation)

- **ReturnSep:** the return for the company's stock during September (in the year of the observation)
- **ReturnOct:** the return for the company's stock during October (in the year of the observation)
- **ReturnNov:** the return for the company's stock during November (in the year of the observation)
- **PositiveDec:** whether or not the company's stock had a positive return in December (in the year of the observation). This variable takes value 1 if the return was positive, and value 0 if the return was not positive.

For the first 11 variables, the value stored is a proportional change in stock value during that month. For instance, a value of 0.05 means the stock increased in value 5% during the month, while a value of -0.02 means the stock decreased in value 2% during the month.

- Load **StocksCluster.csv** into a data frame called **stocks**. How many observations are in the dataset?
- What proportion of the observations have positive returns in December?
- What is the maximum correlation between any two return variables in the dataset? You should look at the pairwise correlations between ReturnJan, ReturnFeb, ReturnMar, ReturnApr, ReturnMay, ReturnJune, ReturnJuly, ReturnAug, ReturnSep, ReturnOct, and ReturnNov
- Which month (from January through November) has the largest mean return across all observations in the dataset? Which month (from January through November) has the smallest mean return across all observations in the dataset?
- Run the following commands to split the data into a training set and testing set, putting 70% of the data in the training set and 30% of the data in the testing set:


```
> set.seed(144)
> spl <- sample.split(stocks$PositiveDec, SplitRatio = 0.7)
> stocksTrain <- subset(stocks, spl == TRUE)
> stocksTest <- subset(stocks, spl == FALSE)
```

 Then, use the stocksTrain data frame to train a logistic regression model (name it StocksModel) to predict PositiveDec using all the other variables as independent variables. What is the overall accuracy on the training set, using a threshold of 0.5? (A possible way to do this is to write a function to calculate accuracy.)
- Now obtain test set predictions from StocksModel. What is the overall accuracy of the model on the test, again using a threshold of 0.5?
- What is the accuracy on the test set of a baseline model that always predicts the most common outcome in the training set?
- Now, let's cluster the stocks. The first step in this process is to remove the dependent variable using the following commands:

```

> limitedTrain <- stocksTrain
> limitedTrain$PositiveDec <- NULL
> limitedTest <- stocksTest
> limitedTest$PositiveDec <- NULL

```

Why do we need to remove the dependent variable in the clustering phase of the cluster-then-predict methodology?

- i. Leaving in the dependent variable might lead to unbalanced clusters
 - ii. Removing the dependent variable decreases the computational effort needed to cluster
 - iii. Needing to know the dependent variable value to assign an observation to a cluster defeats the purpose of the methodology
- (i) In some cases where we have a training and testing set, we might want to normalize by the mean and standard deviation of the variables in the training set. We can do this by using the **caret** package passing just the training set to the `preProcess` function, which normalizes variables by subtracting by the mean and dividing by the standard deviation.
- ```

> library(caret)
> preproc <- preProcess(limitedTrain)
> normTrain <- predict(preproc, limitedTrain)
> normTest <- predict(preproc, limitedTest)

```
- What is the mean of the `ReturnJan` variable in `normTrain`?
- What is the mean of the `ReturnJan` variable in `normTest`?
- (j) Why is the mean `ReturnJan` variable much closer to 0 in `normTrain` than in `normTest`?
- i. Small rounding errors exist in the normalization procedure
  - ii. The distribution of the `ReturnJan` variable is different in the training and testing set
  - iii. The distribution of the dependent variable is different in the training and testing set
- (k) Set the random seed to 144 (it is important to do this again, even though we did it earlier). Run k-means clustering with 3 clusters on `normTrain`, storing the result in an object called `km`. Which cluster has the largest number of observations?
- i. Cluster 1
  - ii. Cluster 2
  - iii. Cluster 3
- (l) In this question, we use the `flexclust` package to obtain training set and testing set cluster assignments for our observations and to do the predictions. Use the following commands:
- ```

> library(flexclust)
> km.kcca <- as.kcca(km, normTrain)
> clusterTrain <- predict(km.kcca)
> clusterTest <- predict(km.kcca, newdata=normTest)

```
- How many test-set observations were assigned to Cluster 2?
- (m) Using the `subset` function, build data frames `stocksTrain1`, `stocksTrain2`, and `stocksTrain3`, containing the elements in the `stocksTrain` data frame assigned to clusters 1,

2, and 3, respectively (be careful to take subsets of `stocksTrain`, not of `normTrain`). Similarly build `stocksTest1`, `stocksTest2`, and `stocksTest3` from the `stocksTest` data frame. Which training set data frame has the highest average value of the dependent variable?

- i. `stocksTrain1`
- ii. `stocksTrain2`
- iii. `stocksTrain3`

- (n) Build logistic regression models `StocksModel1`, `StocksModel2`, and `StocksModel3`, which predict `PositiveDec` using all the other variables as independent variables. `StocksModel1` should be trained on `stocksTrain1`, `StocksModel2` should be trained on `stocksTrain2`, and `StocksModel3` should be trained on `stocksTrain3`. Which variables have a positive sign for the coefficient in at least one of `StocksModel1`, `StocksModel2`, and `StocksModel3` and a negative sign for the coefficient in at least one of `StocksModel1`, `StocksModel2`, and `StocksModel3`?

- (o) Using `StocksModel1`, make test-set predictions called `PredictTest1` on the data frame `stocksTest1`. Using `StocksModel2`, make test-set predictions called `PredictTest2` on the data frame `stocksTest2`. Using `StocksModel3`, make test-set predictions called `PredictTest3` on the data frame `stocksTest3`.

What is the overall accuracy of `StocksModel1` on the test set `stocksTest1`, using a threshold of 0.5?

What is the overall accuracy of `StocksModel2` on the test set `stocksTest2`, using a threshold of 0.5?

What is the overall accuracy of `StocksModel3` on the test set `stocksTest3`, using a threshold of 0.5?

- (p) To compute the overall test-set accuracy of the cluster-then-predict approach, we can combine all the test-set predictions into a single vector and all the true outcomes into a single vector:

```
> AllPredictions <- c(PredictTest1, PredictTest2, PredictTest3)
```

```
> AllOutcomes <- c(stocksTest1$PositiveDec, stocksTest2$PositiveDec, stocksTest3$PositiveDec)
```

What is the overall test-set accuracy of the cluster-then-predict approach, again using a threshold of 0.5?